

Calculadora Neuronal

Seq2seq con atención aditiva de Bahdanau

Una evaluación reproducible de aprendizaje aritmético carácter por carácter, atención dinámica, generalización por longitud y límites adversariales.

Estudiante	Pablo Daniel Barillas Moreno
Carné	22193
Modalidad	Individual
Fecha	2 de octubre de 2026
Repositorio	GitHub · Laboratorio 6

Índice

1 Resumen ejecutivo	2
2 Problema, objetivos y alcance	2
2.1 Pregunta central	2
2.2 Restricciones de la guía	2
3 Fundamento teórico	4
3.1 Del vector fijo al contexto dinámico	4
3.2 Orden de secuencia y distancia de dependencias	4
4 Datos sintéticos y capacidad de memorización	4
5 Arquitectura e implementación	6
5.1 Encoder	6
5.2 Atención	6
5.3 Decoder	6
5.4 Complejidad	6
6 Diseño experimental	7
7 Resultados	8
7.1 Aprendizaje durante 15 épocas	8
7.2 Efecto de la atención por longitud	8
7.3 Prueba de la hipótesis del torneo	10
8 Interpretación de la atención	11
9 Análisis adversarial	12
10 Reproducibilidad y trazabilidad	12
11 Conclusiones	13
Referencias	13

1 Resumen ejecutivo

Este laboratorio estudia si una red neuronal recurrente puede aprender a sumar y restar observando pares de texto, sin recibir reglas aritméticas explícitas. La entrada es una expresión como $4821+3976$; la salida se genera carácter por carácter. Se implementó una arquitectura encoder-decoder con GRU bidireccional, atención aditiva y un decoder autoregresivo. El análisis compara tres modelos: baseline con atención, baseline con contexto fijo y variante de torneo con la salida invertida.

97.20%
validación · torneo

95.00%
cuatro cifras · torneo

186,767
parámetros

La modificación controlada `invertir_salida=True` mejoró 13.50 puntos porcentuales la validación global y 24.67 puntos la exactitud de cuatro cifras respecto del baseline con atención. No incrementó el número de parámetros, ejemplos o épocas. Sin embargo, el modelo obtuvo 0% en cinco y seis cifras y falló en cinco casos válidos cuidadosamente buscados. La conclusión es doble: la inductiva escogida facilita el acarreo dentro de la distribución, pero no produce un algoritmo simbólico universal.

Estado de entrega. El notebook fue ejecutado desde un kernel reiniciado hasta la celda de Entrega. El formulario confirmó el registro y la huella del modelo de torneo se mantuvo en `94e198ec75ec`. La clave del torneo sigue en `None` porque corresponde al Laboratorio 7.

2 Problema, objetivos y alcance

2.1 Pregunta central

La tarea exige transformar una secuencia de caracteres de longitud variable en otra secuencia. Aunque las sumas y restas siguen reglas deterministas, una red entrenada por gradiente solo observa ejemplos. Para resolver correctamente debe representar operandos, operación, posición decimal, acarreos o préstamos y la condición de finalización de la salida.

Los objetivos concretos fueron:

- implementar la atención aditiva de Bahdanau y un paso del decoder;
- comparar atención dinámica contra un único contexto fijo;
- medir exactitud exacta por cantidad de cifras;
- formular y probar una mejora de torneo con una o dos perillas;
- localizar cinco contraejemplos dentro del rango permitido;
- conservar pesos, huellas, métricas, figuras y notebook ejecutado.

2.2 Restricciones de la guía

El entrenamiento usa CPU, PyTorch, Matplotlib y biblioteca estándar. No se utiliza NumPy ni la capa `nn.MultiheadAttention`. Los operandos de entrenamiento no exceden cuatro cifras, la arquitectura

no se modifica en el bloque de torneo y el modelo debe permanecer por debajo de 750,000 parámetros. Las celdas oficiales de verificación se preservaron sin cambios.

Cuadro 1: Matriz de requisitos principales.

Requisito	Evidencia	Estado
Atención aditiva	Formas, normalización y máscara verificadas	Cumple
Paso del decoder	Logits, estado y pesos verificados	Cumple
Exactitud base ≥ 0.60	Exactitud 0.837	Cumple
Máximo 750,000 parámetros	186,767 parámetros	Cumple
Cinco trampas válidas	Cinco fallos reproducidos	Cumple
Registro externo	Mensaje oficial de confirmación visible	Cumple

3 Fundamento teórico

3.1 Del vector fijo al contexto dinámico

En un encoder–decoder clásico, toda la entrada se resume en un vector fijo. Bahdanau, Cho y Bengio identifican este mecanismo como un cuello de botella: a medida que crece la secuencia, el resumen debe preservar demasiados detalles. La atención sustituye esa única decisión global por un contexto específico para cada paso de salida [1].

Sea s el estado actual del decoder y h_j el estado del encoder asociado con el carácter j :

$$e_j = v^\top \tanh(W_q s + W_k h_j), \quad (1)$$

$$\alpha_j = \frac{\exp(e_j)}{\sum_k \exp(e_k)}, \quad (2)$$

$$c = \sum_j \alpha_j h_j. \quad (3)$$

Los puntajes de posiciones de relleno se sustituyen por $-\infty$ antes del *softmax*; por tanto, su peso es exactamente cero. El contexto c permite al decoder consultar distintas regiones de la expresión mientras genera cada dígito.

3.2 Orden de secuencia y distancia de dependencias

Sutskever, Vinyals y Le mostraron que invertir la fuente podía reducir la distancia mínima entre elementos relacionados de entrada y salida y facilitar la optimización [2]. La aritmética decimal sugiere una adaptación más específica: una persona suma desde las unidades y propaga el acarreo hacia la izquierda. Si el decoder escribe la respuesta en orden convencional, debe predecir primero el dígito más significativo, que puede depender de un acarreo originado varias columnas a la derecha.

La hipótesis previa al entrenamiento fue entonces: invertir únicamente la salida hará que el decoder genere unidades primero y almacene el acarreo en su estado recurrente. La entrada se conservó intacta para aislar una sola modificación.

4 Datos sintéticos y capacidad de memorización

Cada ejemplo se genera al azar; los operandos tienen de una a cuatro cifras, la operación es suma o resta y las restas nunca son negativas. El vocabulario tiene 15 símbolos: diez dígitos, dos operadores y tres tokens especiales.

Para dos operandos de exactamente cuatro cifras existen $9,000^2 = 81,000,000$ sumas ordenadas. En restas no negativas, los pares con $a \geq b$ son

$$\frac{9,000(9,000 + 1)}{2} = 40,504,500.$$

El total asciende a 121,504,500 expresiones, frente a solo 40,000 ejemplos de entrenamiento. La muestra representa menos de 0.033% de ese subconjunto; el modelo no puede memorizar exhaustivamente el espacio y debe generalizar regularidades.

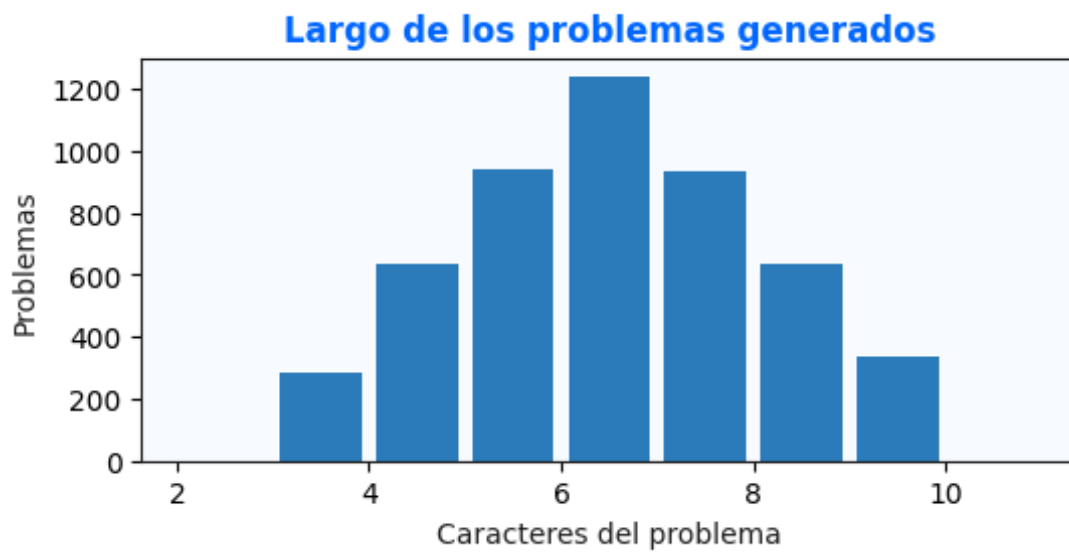


Figura 1: Distribución de longitudes en la validación reproducible.

5 Arquitectura e implementación

5.1 Encoder

El encoder convierte caracteres en embeddings y usa una GRU bidireccional. Se emplea la función `pack_padded_sequence` para impedir que la recurrencia procese el padding. Las dos direcciones se concatenan tanto en las salidas posicionales como en el resumen global; un puente lineal produce el estado inicial del decoder.

5.2 Atención

La implementación usa proyecciones lineales sin sesgo para consulta y claves, activación hiperbólica, proyección escalar, máscara y `torch.bmm` para la suma ponderada. La verificación oficial comprueba:

- pesos de forma (B, T) y contexto de forma (B, d_{enc}) ;
- suma de pesos igual a uno por ejemplo;
- peso exactamente cero en posiciones `<PAD>`.

5.3 Decoder

En cada paso, el token previo se embebe; la atención calcula un contexto desde el estado actual; una `GRUCell` recibe la concatenación $[embedding, contexto]$; y la capa final recibe $[nuevo_estado, contexto]$. Durante el entrenamiento se usa *teacher forcing*; durante la inferencia, el token predicho realimenta el paso siguiente.

5.4 Complejidad

Los tres modelos contienen 186,767 parámetros. El baseline sin atención conserva los módulos de atención aunque no los consulta, por lo que el conteo coincide; esto mantiene la clase y el formato de checkpoint uniformes. La capacidad utilizada equivale a 24.9% del máximo permitido.

6 Diseño experimental

Cuadro 2: Configuración compartida y variable experimental.

	Con atención	Sin atención	Torneo
Ejemplos	40,000	40,000	40,000
Épocas	15	15	15
Dimensión embedding	32	32	32
Dimensión oculta	64	64	64
Tasa de aprendizaje	0.003	0.003	0.003
Semilla	2026	2026	2026
Atención	Sí	No	Sí
Salida invertida	No	No	Sí

La exactitud exige coincidencia completa de la cadena; un solo dígito incorrecto vuelve incorrecto todo el ejemplo. La validación global usa 2,000 problemas. La evaluación por longitud usa 300 problemas para cada nivel de una a seis cifras, donde cinco y seis son estrictamente fuera del rango de entrenamiento.

La comparación del torneo mantiene semilla, muestra, épocas, optimizador y capacidad. Esto fortalece la atribución causal a la inversión de salida, aunque una sola semilla no permite estimar variabilidad entre entrenamientos.

7 Resultados

7.1 Aprendizaje durante 15 épocas

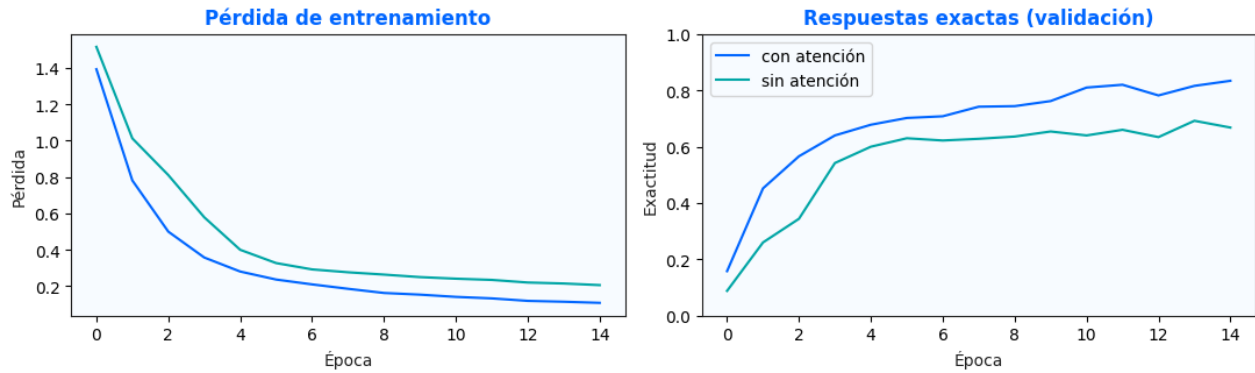


Figura 2: Pérdida y exactitud de validación por época para ambos baselines.

El baseline con atención finalizó con 83.70% de exactitud global, 16.45 puntos por encima del modelo con contexto fijo. Las curvas no son estrictamente monótonas: la exactitud exacta puede fluctuar aunque la pérdida por token disminuya, porque basta un token erróneo para invalidar una respuesta completa.

Cuadro 3: Resultados globales, capacidad e identidad de pesos.

Modelo	Validación	Parámetros	Huella
Base con atención	83.70%	186,767	7dd56f9acf5d
Base sin atención	67.25%	186,767	11a6346d696d
Torneo, salida invertida	97.20%	186,767	94e198ec75ec

7.2 Efecto de la atención por longitud

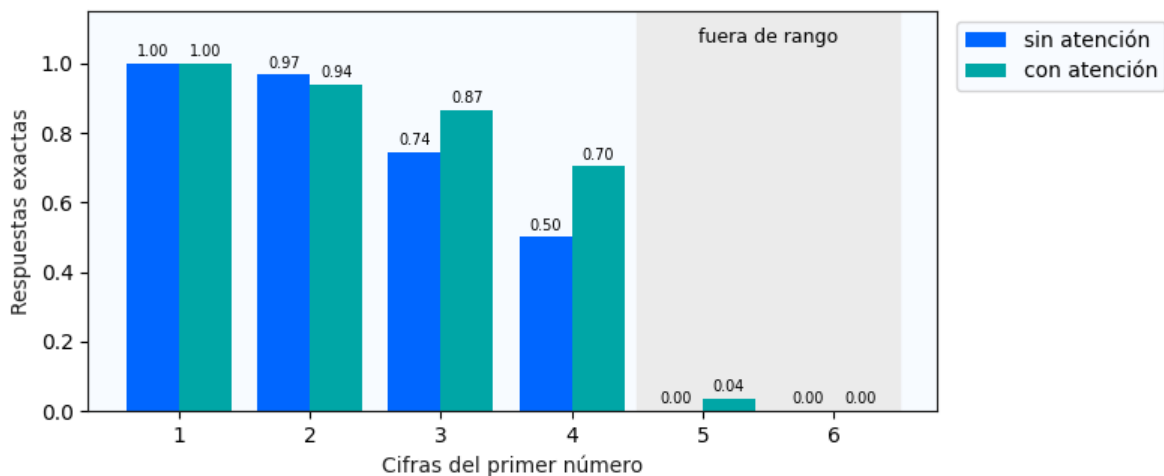


Figura 3: Exactitud del baseline con y sin atención por cantidad de cifras.

Con un dígito no hay diferencia; con dos, el modelo fijo es ligeramente superior. En tres y cuatro cifras, la atención obtiene 86.67% y 70.33%, frente a 74.33% y 50.00%. Esto apoya la explicación del cuello de botella: la selección dinámica se vuelve valiosa conforme crece la secuencia.

7.3 Prueba de la hipótesis del torneo

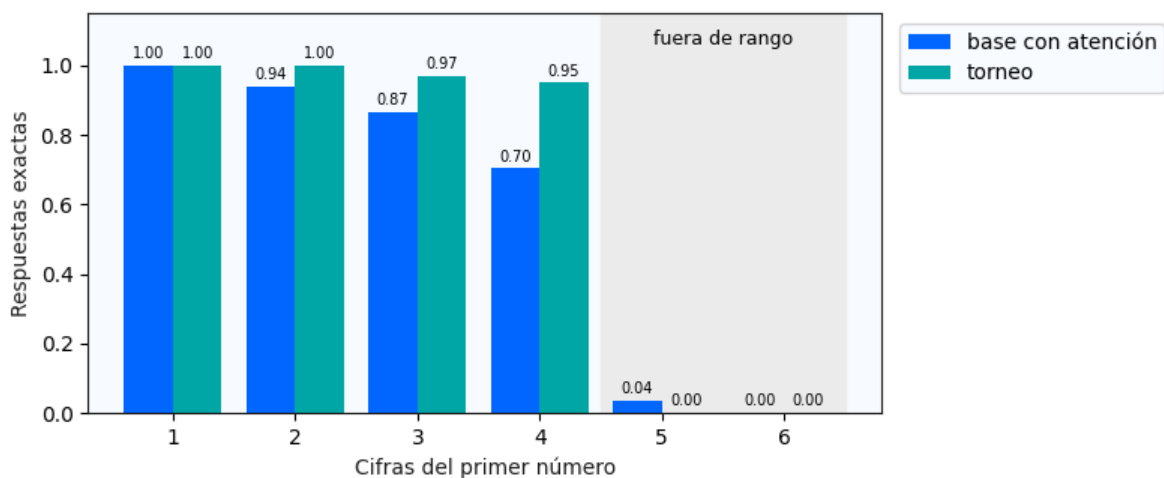


Figura 4: Comparación controlada entre el baseline con atención y la salida invertida.

Cuadro 4: Exactitud exacta por longitud.

Cifras	Con atención	Sin atención	Torneo
1	100.00%	100.00%	100.00%
2	94.00%	96.67%	100.00%
3	86.67%	74.33%	97.00%
4	70.33%	50.00%	95.00%
5, fuera de rango	3.67%	0.00%	0.00%
6, fuera de rango	0.00%	0.00%	0.00%

La hipótesis se cumple: invertir la salida mejora la métrica objetivo y no sacrifica la validación global. La mejora es compatible con una reducción de la dificultad temporal del acarreo. No obstante, el cero por ciento fuera de rango muestra que la intervención mejora interpolación y no extrapolación de longitud.

8 Interpretación de la atención

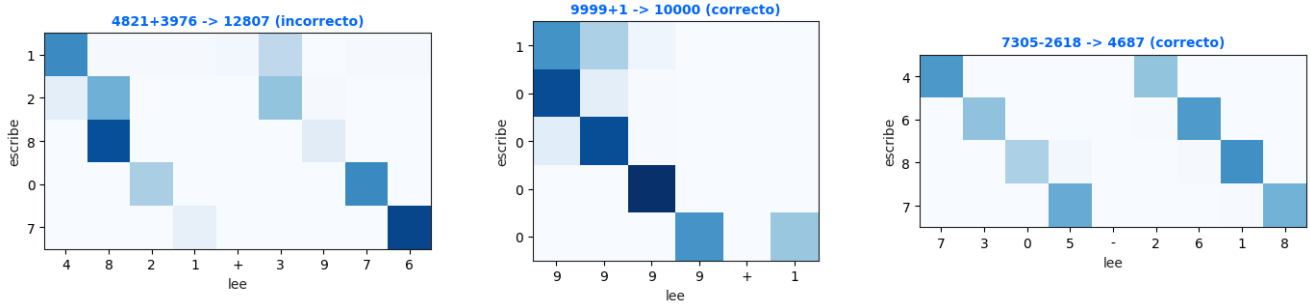


Figura 5: Mapas del baseline con atención para tres expresiones.

En el baseline, el peso cambia entre pasos de salida y se concentra en regiones diferentes de los operandos. En $4821+3976$ se observan alineaciones posicionales, pero no una diagonal perfecta que equivalga a la suma por columnas. El decoder convencional escribe primero la izquierda, mientras el acarreo surge a la derecha, lo que obliga al estado a anticipar dependencias de largo alcance.

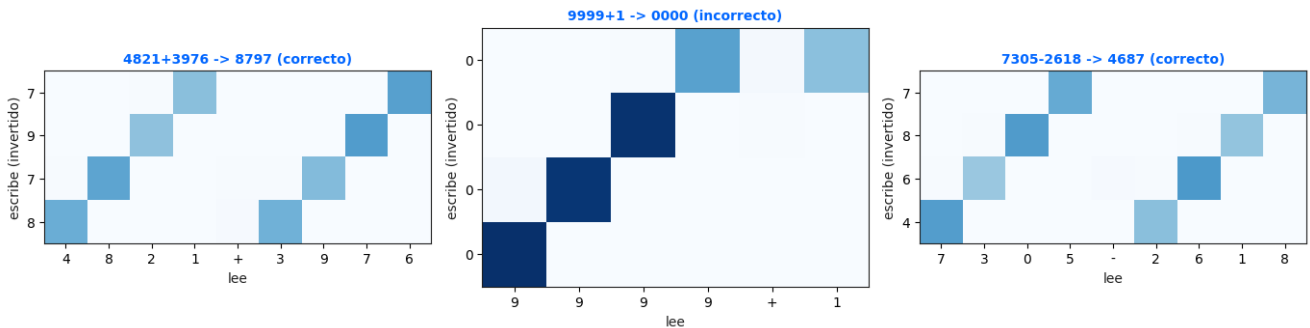


Figura 6: Mapas del modelo de torneo; la salida se genera internamente en orden invertido.

En el modelo de torneo las filas deben leerse en orden de generación interno: unidades antes que cifras de mayor valor. Esta representación temporal es más congruente con el acarreo. Sin embargo, un mapa de atención muestra distribución de pesos, no una prueba causal de que cada celda implemente una operación simbólica.

9 Análisis adversarial

Se exploraron primero familias difíciles —acarreo completo, préstamos a través de ceros, resultados cortos y cambios de longitud— y luego una búsqueda determinista de hasta 15,000 expresiones válidas. Los primeros cinco fallos distintos fueron:

Cuadro 5: Casos que rompen el modelo final dentro del rango permitido.

Problema	Predicción	Correcta	Interpretación
9999+1	0000	10000	Acarreo global y nueva cifra
9998+2	0000	10000	Acarreo desde unidades
9999+9999	18998	19998	Acarreos simultáneos
1000-999	11	1	Préstamo y longitud de salida
4000-3999	001	1	Préstamos y ceros iniciales

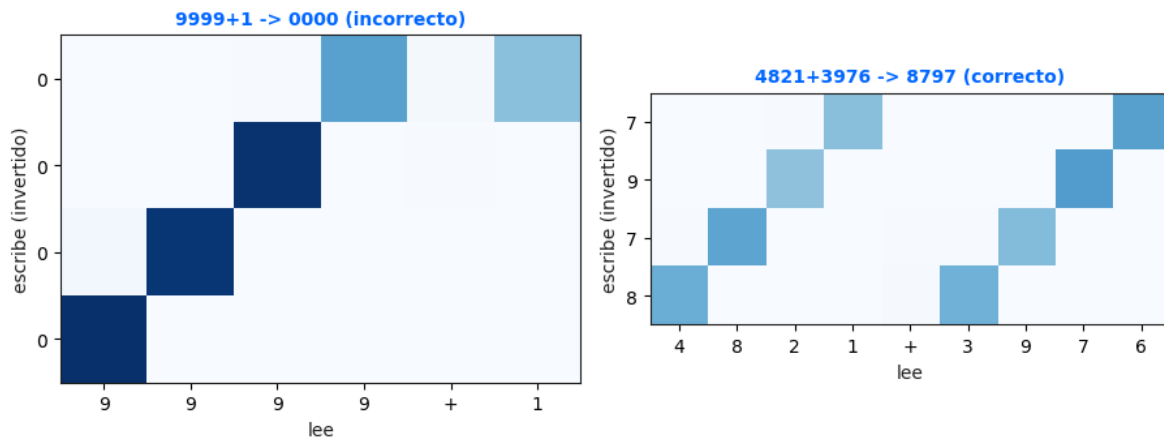


Figura 7: Comparación entre 9999+1 y el caso de control 4821+3976.

El caso 9999+1 exige producir cinco dígitos a partir de operandos de cuatro y propagar un acarreo por todas las columnas. El modelo produce 0000: capta una parte del patrón local, pero omite la nueva cifra significativa. La evidencia respalda que aprendió a imitar muchas operaciones dentro de la distribución, no una regla universal. Un único contraejemplo basta para refutar universalidad; aquí se presentan cinco.

10 Reproducibilidad y trazabilidad

El repositorio conserva el machote original, el notebook final con salidas, tres checkpoints, métricas JSON, figuras exportadas y documentación por tarea. Los scripts reconstruyen el notebook, ejecutan dos etapas sin enviar el formulario, exportan figuras y verifican la entrega.

```
python scripts/build_notebook.py
python scripts/execute_notebook.py --stage discover
python scripts/build_notebook.py
python scripts/execute_notebook.py --stage final
```

```
python scripts/exportar_figuras.py
python scripts/verificar_entrega.py
```

El notebook contiene 15 celdas de código ejecutadas con conteos consecutivos y sin errores hasta la entrega. La salida del formulario confirma el envío y repite la huella 94e198ec75ec. La única celda sin ejecutar es la del torneo; CLAVE=None se mantiene hasta que el profesor revele la clave del Laboratorio 7.

La metodología de producción y revisión de visualizaciones siguió principios de evidencia, legibilidad y reproducción asistida descritos por Kassis et al. [3]; esa referencia documenta el proceso, no sustituye las fuentes académicas de atención y seq2seq.

11 Conclusiones

1. La atención aporta una ventaja clara cuando crece la longitud dentro de la distribución: 20.33 puntos en cuatro cifras respecto del contexto fijo.
2. Invertir la salida es una intervención pequeña y eficaz: lleva la exactitud de cuatro cifras a 95.00% y la validación global a 97.20% sin sumar parámetros.
3. La visualización de atención ayuda a inspeccionar alineaciones, pero no demuestra razonamiento simbólico ni causalidad interna.
4. La generalización de longitud es deficiente: el modelo de torneo cae a 0% con cinco y seis cifras.
5. Los fallos en acarreo globales y préstamos entre ceros prueban que una métrica promedio alta debe complementarse con evaluación adversarial.

Como trabajo futuro, se recomienda repetir el experimento con múltiples semillas, separar conjuntos por familias de acarreo y préstamo, evaluar exactitud por token además de cadena completa y estudiar currículos que aumenten progresivamente la longitud. Cualquier cambio para el torneo deberá respetar la arquitectura, el máximo de cuatro cifras durante entrenamiento y la huella ya registrada.

Referencias

- [1] D. Bahdanau, K. Cho y Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv:1409.0473, 2014. <https://arxiv.org/abs/1409.0473>
- [2] I. Sutskever, O. Vinyals y Q. V. Le. *Sequence to Sequence Learning with Neural Networks*. arXiv:1409.3215, 2014. <https://arxiv.org/abs/1409.3215>
- [3] T. Kassis, V. Agarwal, Y. He, D. Patel y A. M. Brueckner. *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. arXiv:2609.00065, 2026. <https://doi.org/10.48550/arXiv.2609.00065>

Declaración de apoyo con IA

Se utilizó IA como apoyo de ingeniería para estructuración, revisión, automatización y presentación. Las afirmaciones académicas se sustentan en las fuentes citadas; la IA no se considera una fuente. Todas las métricas, predicciones, huellas y figuras reportadas provienen de la ejecución reproducible incluida.